

Section 1: Advanced Java Core & JVM Mechanics

Q1. If a Java application experiences a high rate of object allocation and sudden, brief spikes in latency, which memory region's sizing should you inspect first?

- A. Metaspace
- B. Survivor Spaces (s0/s1) inside the Young Generation
- C. Code Cache
- D. Off-heap Direct Memory

Q2. What is the fundamental functional difference between `java.util.Optional.map()` and `java.util.Optional.flatMap()`?

- A. `map()` can return `null`, while `flatMap()` throws a `NullPointerException` if the value is missing.
- B. `map()` expects a mapping function that returns a raw value, while `flatMap()` expects a function that returns another `Optional`.
- C. `map()` executes synchronously, whereas `flatMap()` executes asynchronously on a separate worker thread.
- D. `map()` modifies the original `Optional` object, while `flatMap()` creates a shallow copy.

Q3. Consider the following code snippet. What is printed?

```
java
List<String> list = new ArrayList<>(List.of("A", "B"));
for (String s : list) {
    if (s.equals("A")) {
        list.remove(s);
    }
}
System.out.println(list);
Use code with caution.
```

- A. [B]
- B. [A, B]
- C. Throws `ConcurrentModificationException` at runtime.
- D. Throws `UnsupportedOperationException` at runtime.

Q4. What occurs when a Java thread triggers a `StackOverflowError` versus an `OutOfMemoryError`: Java heap space?

- A. `StackOverflowError` means the Metaspace is full; `OutOfMemoryError` means the physical RAM is exhausted.
- B. `StackOverflowError` occurs when thread execution frames exceed allocated thread stack size; `OutOfMemoryError` occurs when new object allocations exceed available heap space.
- C. `StackOverflowError` applies only to recursive functions; `OutOfMemoryError` applies only to `HashMap` instances.
- D. The JVM automatically recovers from a `StackOverflowError`, but crashes instantly on an `OutOfMemoryError`.

Q5. Why does the Java compilation process implement "Type Erasure" for Generics?

- A. To ensure backwards compatibility with older Java versions that did not support generics.
- B. To force the JVM to run faster by deleting variable definitions.
- C. To prevent developers from creating custom collections at runtime.
- D. To automatically convert all reference objects into primitive types.

Q6. What is the true behavior of the `System.gc()` method call in an enterprise Java production environment?

- A. It instantly forces a full, stop-the-world Garbage Collection cycle across all memory regions.
- B. It acts purely as a hint/suggestion to the JVM, which the underlying execution engine can freely ignore.
- C. It frees up memory exclusively in the Metaspace region.
- D. It throws a security exception if the application is running inside a Docker container.

Q7. What is the primary operational mechanism behind Java's Dynamic Proxies (`java.lang.reflect.Proxy`)?

- A. They manipulate raw byte code directly inside the target system files.
- B. They create an on-the-fly implementation of specific interfaces at runtime.
- C. They require target classes to extend an abstract parent base class.
- D. They bypass classloaders to inject instructions straight into CPU registers.

Q8. How does the `String.hashCode()` method calculate its value to minimize collisions?

- A. It returns the internal virtual memory address of the string object.
- B. It uses a polynomial hash function with a prime multiplier constant of 31.
- C. It uses a random number generator seeded with the current system millisecond time.
- D. It calculates the total number of characters multiplied by the string's length.

Q9. Which reference type in Java allows its referent to be cleared by the garbage collector only when the JVM is running critically out of memory?

- A. `WeakReference`
- B. `SoftReference`
- C. `PhantomReference`
- D. Strong Reference

Q10. What is the output of evaluating `0.1 + 0.2 == 0.3` in standard Java using primitive `double` types?

- A. `true`
- B. `false`
- C. Compilation Error
- D. Throws `ArithmeticException`

Section 2: Advanced Concurrency & Parallel Frameworks

Q11. When using a `ThreadPoolExecutor`, what happens when the active thread count reaches the `corePoolSize` and a new task is submitted?

- A. The executor immediately creates a new thread up to the `maximumPoolSize`.
- B. The task is placed into the executor's defined `BlockingQueue`.
- C. The task is instantly rejected, throwing a `RejectedExecutionException`.
- D. The main thread blocks until an active pool thread becomes available.

Q12. What is a "Livelock" scenario between two concurrent threads?

- A. Both threads are blocked indefinitely waiting for a lock held by the other.
- B. Threads actively change their internal state in response to each other, but make zero functional forward progress.
- C. A thread is permanently denied execution time because higher priority threads consume all CPU cycles.
- D. A thread terminates prematurely due to an unhandled runtime runtime error flag.

Q13. How does the `CompletableFuture.thenCompose()` method differ fundamentally from `CompletableFuture.thenApply()` ?

- A. `thenCompose()` runs sequentially, while `thenApply()` runs in parallel.
- B. `thenCompose()` flattens nested `CompletableFuture` structures (similar to `flatMap`), while `thenApply()` returns a nested `CompletableFuture` if the mapping function returns a future.
- C. `thenCompose()` accepts a consumer, while `thenApply()` accepts a predicate logic gate.
- D. `thenCompose()` can only be used with HTTP clients; `thenApply()` handles local memory tasks.

Q14. What core design mechanism allows the `ForkJoinPool` (used by parallel streams) to optimize processing efficiency across multi-core processors?

- A. Global synchronization locks across all execution queues.
- B. A Work-Stealing algorithm where idle worker threads steal tasks from the back of busy threads' double-ended queues.
- C. Converting all asynchronous tasks into native database triggers.
- D. Pinning individual threads to specific physical CPU cores permanently.

Q15. In Java's explicit locking framework, what unique advantage does `StampedLock` offer over a standard `ReentrantReadWriteLock`?

- A. It supports an "Optimistic Read" mode that allows reading data without acquiring a pessimistic lock, avoiding thread starvation.
- B. It automatically scales up to support distributed machines over network firewalls.
- C. It completely prevents deadlocks from occurring regardless of how locks are written.
- D. It requires 90% less heap memory allocation to operate.

Section 3: Deep-Dive Spring Boot Framework Internals

Q16. What is the exact execution order of Spring Bean lifecycle callback methods during initialization?

- A. `@PostConstruct` $\rightarrow$ `InitializingBean.afterPropertiesSet()` $\rightarrow$ Custom `init()` method.
- B. Custom `init()` method $\rightarrow$ `@PostConstruct` $\rightarrow$ `InitializingBean.afterPropertiesSet()`.
- C. `InitializingBean.afterPropertiesSet()` $\rightarrow$ Custom `init()` method $\rightarrow$ `@PostConstruct`.
- D. `@PostConstruct` $\rightarrow$ Custom `init()` method $\rightarrow$ `InitializingBean.afterPropertiesSet()`.

Q17. How can you instruct Spring Boot to conditionally instantiate a configuration class bean only if a specific property exists and matches a target value in `application.yml`?

- A. `@ConditionalOnClass`
- B. `@ConditionalOnProperty(name = "target.prop", havingValue = "true")`
- C. `@Profile("production")`
- D. `@RequiredPropertyMatch`

Q18. If a Spring bean is configured with the `prototype` scope and is injected into a `singleton` scope bean, what occurs when a method calls the prototype bean?

- A. A brand-new instance of the prototype bean is generated on every single method call.
- B. Only one instance of the prototype bean is created during application startup and it remains static for the lifecycle of the singleton.
- C. Spring throws a `BeanCreationException` due to conflicting scope declarations.
- D. The prototype bean is automatically converted into a singleton bean context scope.

Q19. To solve the problem described in Q18 and get a fresh instance of the prototype bean every time inside a singleton, which mechanism should be applied?

- A. Marking the singleton bean with `@Scope("lazy")`.
- B. Using Spring's `@Lookup` method injection or an `ObjectProvider` interface.
- C. Declaring the prototype bean field as `volatile`.
- D. Moving the properties definitions into an external text file.

Q20. When implementing Spring Boot Security with JWT, what is the role of the `SecurityContextHolder`?

- A. To store token validation cryptographic keys securely in the database.
 - B. To associate the authenticated user's `Authentication` details with the currently executing execution thread via a `ThreadLocal` strategy.
 - C. To encrypt all traffic entering the network infrastructure gateway.
 - D. To manage session token expiration durations on the client device.
-

Section 4: Data Layer, JPA & Transaction Handling

Q21. What happens if a Spring method annotated with

@Transactional(propagation = Propagation.REQUIRES_NEW) is invoked?

- A. It forces the current running transaction to fail immediately and rolls back data alterations.
- B. It suspends any existing active transaction and opens up a brand-new, completely independent database transaction context.
- C. It joins the outer transaction structure but applies strict isolation logic rules to it.
- D. It forces the database to lock the entire targeted table using exclusive write locks.

Q22. What design risk is introduced when using the @DynamicUpdate annotation on a Hibernate entity?

- A. It forces Hibernate to execute queries sequentially on a single thread.
- B. It incurs a runtime performance overhead because Hibernate must track property changes and dynamically generate the SQL string at runtime instead of using cached statements.
- C. It breaks the entity's direct mapping linkage to relational tables.
- D. It automatically disables caching mechanisms across all secondary tables.

Q23. In Spring Data JPA, what is the primary consequence of failing to provide a @Modifying annotation over a custom @Query that performs an UPDATE or DELETE statement?

- A. The query will run but will return modified records instead of match metrics.
- B. Spring throws an `InvalidDataAccessApiUsageException` or `TransactionRequiredException` at runtime.
- C. The database instance will drop the target index parameters instantly.
- D. Data gets modified, but the persistent cache fails to synchronize properly.

Q24. What performance issue does the "Hibernate First-Level Cache" solve, and what is its lifespan scope?

- A. It buffers query statements globally across the entire cluster lifecycle.
- B. It caches entity objects within the scope of a single Persistence Context (Session), preventing redundant database reads for the same entity in a single transaction.
- C. It speeds up file downloads by mapping data streams to temporary system memory pools.
- D. It replaces the local Redis deployment completely for caching hot database rows.

Q25. What is the fundamental difference between pessimistic locking (@Lock (LockModeType.PESSIMISTIC_WRITE)) and optimistic locking (@Version) in enterprise database interactions?

- A. Pessimistic locks lock rows directly at the database level; Optimistic locks use application-level version checks to detect concurrent modifications without locking rows early.
- B. Pessimistic locking is used exclusively for NoSQL stores; Optimistic locking applies to relational databases.

- C. Optimistic locking guarantees that no application errors can ever happen during data writes.
 - D. Pessimistic locking reduces query execution latency by 50% under massive concurrent loads.
-

Section 5: Enterprise Microservices & Cloud Patterns

Q26. When designing a distributed microservice mesh, how does the "Saga Pattern" handle data failures across decoupled microservice nodes?

- A. By rolling back transactions across all remote servers using a heavy distributed lock coordinator.
- B. By executing explicit "Compensating Transactions" that reverse the business impact of previously successful steps in reverse sequence.
- C. By wiping the local cache configurations and retrying the initial client request indefinitely.
- D. By forcing the microservices mesh to isolate all traffic routes globally.

Q27. When scaling out an API Gateway to handle high-throughput traffic, which non-blocking architecture forms the baseline for Spring Cloud Gateway?

- A. Apache Tomcat Thread-per-request model
- B. Project Reactor and Netty server runtime engine
- C. Virtual basic system file streams execution paths
- D. Distributed enterprise service bus topology structures

Q28. What problem arises if multiple Kafka consumer instances in the exact same Consumer Group are assigned to a topic partition with only 1 partition?

- A. Messages are duplicated and processed by all consumer instances inside that group.
- B. Only one consumer instance is assigned the partition to process messages; the other consumer instances remain idle.
- C. The Kafka broker crashes due to partition assignment conflicts.
- D. Messages are round-robbined between consumers, breaking absolute strict sequence ordering.

Q29. What is the distinct operational difference between a "Filter" and an "Interceptor" within a Spring Boot web application?

- A. Filters belong to the Servlet container and execute before reaching the Spring context; Interceptors are part of the Spring MVC framework and execute between Handler Mappings and Controllers.
- B. Filters only evaluate JSON data streams; Interceptors evaluate pure binary attachments.
- C. Interceptors bypass application security validation protocols completely.
- D. Filters cannot access the HTTP response payload under any circumstance.

Q30. Under what microservice failure scenario would you choose to configure a "Bulkhead Pattern"?

- A. When network hackers perform Denial-of-Service attacks against server gateways.
- B. To isolate resources (like dedicated thread pools or connections) so that a failure in one downstream service does not exhaust all system resources and bring down the entire application.
- C. To encrypt sensitive financial data payloads moving through data transport lines.
- D. To format localized logging statements neatly before exporting profiles to monitoring suites.

Complete Answer Key & Deep Technical Clarifications

Q#	Key	Core Technical Clarification
1	B	If short-lived objects spill over from the Young Gen Eden space and overwhelm the Survivor spaces (S0/S1), they are prematurely promoted to the Old Gen. This triggers frequent, costly Minor/Major GC pauses, causing latency spikes.
2	B	<code>map()</code> automatically wraps the output inside an <code>Optional</code> . If your function already returns an <code>Optional</code> , <code>map()</code> leaves you with a nested <code>Optional<Optional<T>></code> . <code>flatMap()</code> keeps the structure flat as an <code>Optional<T></code> .
3	C	The enhanced for-loop uses an underlying <code>Iterator</code> . Modifying the list directly via <code>list.remove()</code> instead of <code>iterator.remove()</code> desynchronizes the internal modification counter (<code>modCount</code>), triggering a <code>ConcurrentModificationException</code> .
4	B	Each thread gets its own execution stack space. Pushing too many method execution frames onto this stack (e.g., deep recursion) causes a <code>StackOverflowError</code> . Real objects live on the shared heap; when it is full, you get an <code>OutOfMemoryError</code> .
5	A	Generics were introduced in Java 5. To ensure compiled code could run seamlessly on older JVM specs without breaking legacy libraries, the compiler completely strips type parameters (erases them) during compile time, replacing them with bounds or <code>Object</code> .
6	B	Calling <code>System.gc()</code> runs code that suggests the JVM clean memory. The JVM treats this as a loose hint and can choose to ignore it entirely. In production frameworks, this flag is often disabled globally via <code>-XX:+DisableExplicitGC</code> .
7	B	Java's core dynamic proxies use core interfaces via reflection wrappers to proxy calls. If a class lacks explicit interfaces, bytecode engineering tools (like CGLIB or ByteBuddy) must be used to create a subclass instead.
8	B	The value 31 is an odd prime. It minimizes hashing bucket distribution collisions and allows compilers to optimize multiplication into a fast bitwise shift and subtraction (<code>31 * i == (i << 5) - i</code>) for hardware efficiency.
9	B	<code>SoftReference</code> instances are highly persistent compared to <code>WeakReference</code> configurations. The Garbage Collector will pass over them unless it risks throwing an imminent <code>OutOfMemoryError</code> , making them ideal for memory-sensitive caches.
10	B	Primitive float/double data types use IEEE 754 floating-point math binary specifications. Fractions like 0.1 and 0.2 cannot be represented with perfect accuracy in binary, leading to rounding errors (<code>\(0.1 + 0.2\)</code> evaluates to <code>\(0.30000000000000004\)</code>).

11 B	When the core pool fills up, tasks queue up in the <code>BlockingQueue</code> instead of spawning threads. The pool will only scale up to its <code>maximumPoolSize</code> once the underlying queue capacity is completely exhausted.
12 B	In a deadlock, threads are frozen and waiting. In a livelock, threads are actively running, changing state, and consuming CPU cycles, but they are stuck in a polite loop of yielding to one another, preventing actual forward progress.
13 B	<code>thenApply</code> takes a function returning a value <code>T</code> and outputs a <code>CompletableFuture<T></code> . If that function returns a <code>CompletableFuture<U></code> , you get a nested future. <code>thenCompose</code> flattens it, returning a clean, single-layer <code>CompletableFuture<U></code> .
14 B	The <code>ForkJoinPool</code> creates a queue for each thread. When a thread finishes its own tasks, it minimizes CPU idle time by navigating to the end of another thread's queue to steal work, reducing thread contention.
15 A	A <code>StampedLock</code> provides an optimistic read mode. It gives you a non-blocking stamp to read data without acquiring a read lock. It then validates the stamp afterward; if a write occurred during the read, it falls back to a pessimistic lock.
16 A	Spring follows a strict lifecycle path during initialization. It executes JSR-250 <code>@PostConstruct</code> annotations first, then checks for the <code>InitializingBean</code> callback interface, and finally runs any custom declared <code>init-method</code> .
17 B	The <code>@ConditionalOnProperty</code> annotation evaluates properties in <code>application.properties/yml</code> . It ensures a bean is only wired into the application context if the specified property key matches the required target value.
18 B	A singleton bean is instantiated only once during application startup. Because its dependencies are injected exactly once during creation, it will capture and hold onto that initial prototype instance forever, rendering the prototype scope ineffective.
19 B	Since a singleton cannot re-inject fields dynamically, using <code>@Lookup</code> instructs Spring to override the method and dynamically fetch a fresh instance from the application context every single time that specific method is called.
20 B	The <code>SecurityContextHolder</code> serves as the central security registry for the current request thread. It binds the authenticated identity to a <code>ThreadLocal</code> storage object, making it accessible throughout the down-stream execution flow.
21 B	<code>REQUIRES_NEW</code> strictly decouples transaction scopes. It pauses the current transaction and opens a clean database connection transaction. A failure or rollback in the outer scope will not impact data committed by this new transaction.
22 B	Hibernate typically generates and caches basic SQL <code>UPDATE</code> strings for entities at startup. <code>@DynamicUpdate</code> forces it to evaluate which fields actually changed at runtime to build a custom SQL string every time, increasing CPU overhead.
23 B	Spring Data repositories expect queries to be data fetchers (<code>SELECT</code>). For data modification queries (<code>UPDATE/DELETE</code>), you must use <code>@Modifying</code> to instruct the framework to execute the query via <code>executeUpdate()</code> instead of <code>executeQuery()</code> .
24 B	Hibernate's First-Level Cache is bound directly to the active <code>Session</code> instance. It caches objects within the current transaction scope, preventing duplicate database hits if the same record is looked up multiple times in the same request.
25 A	Pessimistic locking uses database features like <code>SELECT ... FOR UPDATE</code> to block other transactions from reading or modifying rows. Optimistic locking avoids database-level locks, using a <code>@Version</code> field to throw an exception if a concurrent update is detected.
26 B	Sagas abandon absolute ACID guarantees across distributed microservices. Instead of using complex global locks, each microservice step executes locally. If a step fails downstream, the orchestrator triggers compensating actions to undo the previous steps.

- | | |
|------|---|
| 27 B | Classic Tomcat configurations assign a dedicated physical thread to each HTTP request. Spring Cloud Gateway leverages non-blocking, event-driven loops powered by Netty and Project Reactor, allowing a few core threads to handle thousands of concurrent connections. |
| 28 B | Kafka enforces strict ordering by assigning a single partition to exactly one consumer within a specific consumer group. If you have more consumers than partitions in a group, the extra consumer instances will remain idle. |
| 29 A | Servlet Filters sit on the outer edge of the application and intercept requests before they reach Spring's <code>DispatcherServlet</code> . Interceptors live inside the Spring MVC ecosystem, giving them access to target handlers, controllers, and Spring AOP contexts. |
| 30 B | The Bulkhead pattern isolates your system resources into bounded pools (similar to the watertight compartments of a ship). If a downstream dependency slows down, it only exhausts its own dedicated pool, keeping the rest of the application responsive. |